Kirchhoff-Love plate with Hellan-Herrmann-Johnson method

Contents

In this section we discretize and solve the Kirchhoff-Love plate equation (see Reissner-Mindlin and Kirchhoff-Love plates), which is a fourth order problem,

$$\int_\Omega \mathbb{D}\nabla^2 w : \nabla^2 \delta w \, dx = \int_\Omega f \, \delta w \, dx, \qquad \forall \delta w,$$

where $\mathbb{D}\varepsilon = \frac{Et^3}{12(1-\nu^2)}((1-\nu)\varepsilon + \nu \operatorname{tr}(\varepsilon) I_{2\times 2})$ is the material tensor, E and ν are the Young's modulus and Poisson ratio. The thickness parameter t gets sometimes absorbed by the right-hand side f by dividing the equation by t^3.

The problem is well-posed for $w \in H^2(\Omega)$ (Lax-Milgram) and reads in strong form

$$\operatorname{divdiv}(\mathbb{D}\nabla^2 w) = f \qquad + \text{bc.}$$

Due to the increased regularity of w point forces f are well-defined. To solve the Kirchhoff-Love plate equation with a conforming Galerkin method in the elliptic setting would require H^2-conforming finite elements, where the derivatives are also continuous over elements. Also (hybrid) Discontinuous-Galerkin methods NGSolve docu - Fourth order equation are possible, but would need a high polynomial degree.

8.3.1. Hellan-Herrmann-Johnson method

Instead we use the Hellan-Herrmann-Johnson (HHJ) method for Kirchhoff-Love plates. After some decades the method regained huge interest. See e.g. [Hellan 67, Herrmann 67, Johnson 73, Arnold+Brezzi 85, Comodi 89, Blum+Rannacher 90, Stenberg 91, Krendl+Rafetseder+Zulehner 16, Chen+Hu+Huang 16, Braess+Pechstein+Schöberl 17].

We rewrite the fourth order problem into a second order mixed saddle point problem by introducing the linearized bending moment tensor $\sigma = \mathbb{D}\nabla^2 w$ in the $H(\mathrm{divdiv}) = \{\sigma \in L^2(\Omega, \mathbb{R}^{2\times 2}_{\mathrm{sym}}) \,|\, \mathrm{divdiv}\sigma \in H^{-1}(\Omega)\}$ space as additional unknown leading to

$$
\begin{aligned}
\int_\Omega \mathbb{D}^{-1}\sigma : \tau \, dx - \langle \tau, \nabla^2 w\rangle &= 0 && \forall \tau \in H(\mathrm{divdiv}),\\
-\langle \sigma, \nabla^2 v\rangle &= -\int_\Omega f\, v \, dx && \forall v \in H^1,
\end{aligned}
$$

where $\mathbb{D}^{-1}\varepsilon = \frac{12(1-\nu^2)}{Et^3}\left(\frac{1}{1-\nu}\varepsilon - \frac{\nu}{1-\nu^2}\mathrm{tr}(\varepsilon) I_{2\times 2}\right)$ and the duality pairing $\langle \sigma, \nabla^2 w\rangle$ is defined on a triangulation $\mathcal{T}$ of Ω similar to the TDNNS pairing by

$$
\begin{aligned}
\langle \sigma, \nabla^2 w\rangle &= \sum_{T\in\mathcal{T}} \int_T \sigma : \nabla^2 w \, dx - \int_{\partial T} \sigma_{nn} \frac{\partial w}{\partial n}\, ds \\
&= -\sum_{T\in\mathcal{T}} \int_T \mathrm{div}(\sigma)\cdot \nabla w \, dx + \int_{\partial T} \sigma_{nt} \frac{\partial w}{\partial t}\, ds = -\langle \mathrm{div}\sigma, \nabla w\rangle.
\end{aligned}
$$

Here ∂T denotes the element-boundary (edges) of T, n the normal vector and t the tangential vector on ∂T. With e.g. $\sigma_{nt} := t^\top \sigma n$ we denote the normal tangential component of σ.

We consider a cross plate, which is clamped at the top and bottom edges and a force is applied on the right boundary. Setting $\sigma_{nn} = 0$ corresponds to prescribing no stress condition on the boundary being part of the free boundary conditions. Setting $w \in H^1_0(\Omega)$ together with $\sigma_{nn} = 0$ would give a simply-supported plate. Setting $w \in H^1(\Omega)$ with homogeneous Neumann data for σ the displacement is free, but the plate cannot ''rotate'' at the boundary.

```
from ngsolve import *
```

```
from ngsolve.webgui import Draw
from netgen.occ import *

L = 0.5

wp = WorkPlane().MoveTo(-0.5 * L, -0.5 * L)
for i in range(4):
    wp.Rotate(-90).Line(L).Rotate(90).Line(L / 4).Rotate(90).Line(L)
shape = wp.Face()
shape.edges.name = "free"
shape.edges.Max(X).name = "force"
shape.edges.Max(Y).name = "clamped"
shape.edges.Min(Y).name = "clamped"
mesh = Mesh(OCCGeometry(shape, dim=2).GenerateMesh(maxh=0.05))
Draw(mesh);
```

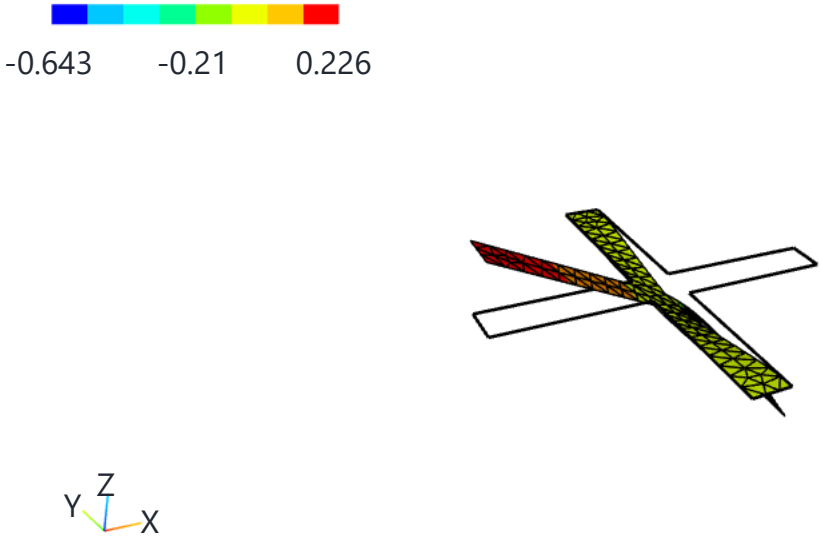
Open Controls

NGSolve 6.2.2404-22-g3e237987d

```
E = 2e3
nu = 0.3
thickness = 0.1
force = 1


def DMatInv(mat, E, nu):
    return (
```

```
        12
        * (1 - nu**2)
        / (thickness**3 * E)
        * (1 / (1 - nu) * mat - nu / (1 - nu**2) * Trace(mat) * Id(2))
    )


def KirchhoffLovePlateHHJ(order=1):
    V = HDivDiv(mesh, order=order - 1, dirichlet="free|force")
    Q = H1(mesh, order=order, dirichlet="clamped")
    X = V * Q
    (sigma, w), (tau, v) = X.TnT()

    n = specialcf.normal(2)

    def tang(u):
        return u - (u * n) * n

    a = BilinearForm(X, symmetric=True)
    a += (
        InnerProduct(DMatInv(sigma, E, nu), tau)
        + div(sigma) * Grad(v)
        + div(tau) * Grad(w)
    ) * dx - (sigma[n, :] * tang(Grad(v)) + tau[n, :] * tang(Grad(w))) * dx(
        element_boundary=True
    )
    a.Assemble()

    f = LinearForm(v * force * ds("force")).Assemble()

    gf_solution = GridFunction(X)
    gf_solution.vec.data = a.mat.Inverse(X.FreeDofs(), inverse="") * f.vec
    return gf_solution


gf_solution_ref = KirchhoffLovePlateHHJ(order=3)
gf_sigma_ref, gf_w_ref = gf_solution_ref.components
Draw(gf_sigma_ref, mesh, name="sigma")
Draw(gf_w_ref, mesh, name="disp", deformation=True, euler_angles=[-60, 5, 30]);
```

NGSolve 6.2.2404-22-g3e237987d



NGSolve 6.2.2404-22-g3e237987d

8.3.2. Hybridization of Hellan-Herrmann-Johnson method

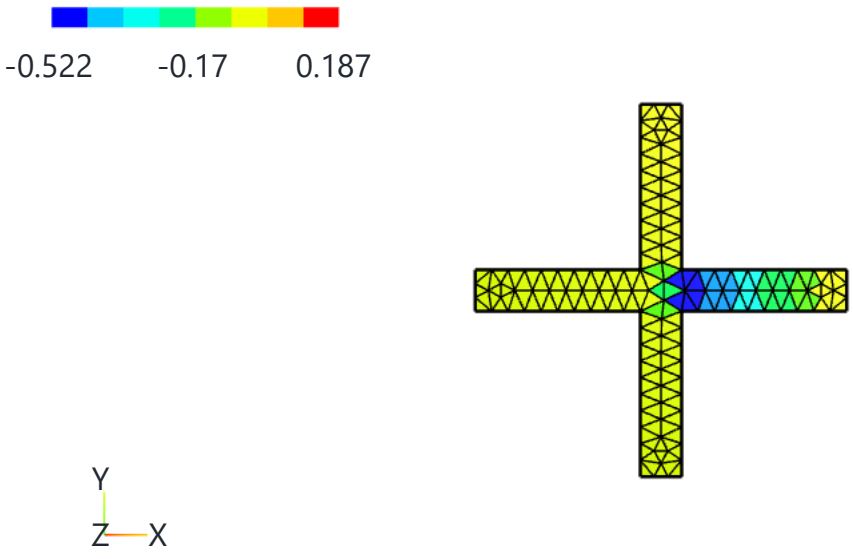
A drawback of the HHJ method the current form is that it is a saddle point problem. To regain a positive definite system we can use hybridization techniques NGSolve docu - Static condensation by breaking the normal-normal continuity of σ and reinforcing it by means of a normal-continuous Lagrange multiplier $\alpha \in \Lambda$

$$
\begin{aligned}
\int_\Omega \mathbb{D}^{-1}\sigma : \tau \, dx - \langle \tau, \nabla^2 w \rangle \qquad + \sum_{E \in \mathcal{E}} \int_E [\![\tau_{nn}]\!] \alpha_n \, ds &= 0 && \forall \tau \in H^{\mathrm{dc}}(\\
- \langle \sigma, \nabla^2 v \rangle &= - \int_\Omega f \, v \, dx && \forall v \in H^1, \\
\sum_{E \in \mathcal{E}} \int_E [\![\sigma_{nn}]\!] \delta_n \, ds &= 0 && \forall v \in \Lambda.
\end{aligned}
$$

This enables us to statically condense out σ and the resulting system in (u, α) is again symmetric positive definite such that we can use e.g. sparsecholesky solver or CG. The resulting degrees of freedom are equivalent to the famous Morley triangle [Morley. The constant-moment plate-bending element. *Journal of Strain Analysis 6* , 1 (1971), 20-24] which is a non-conforming element for the fourth order plate problem. The physical meaning of α is the normal derivative of the displacement $\alpha_n \hat{=} \frac{\partial w}{\partial n}$. Note, that the essential and natural boundary conditions from σ to α swap.

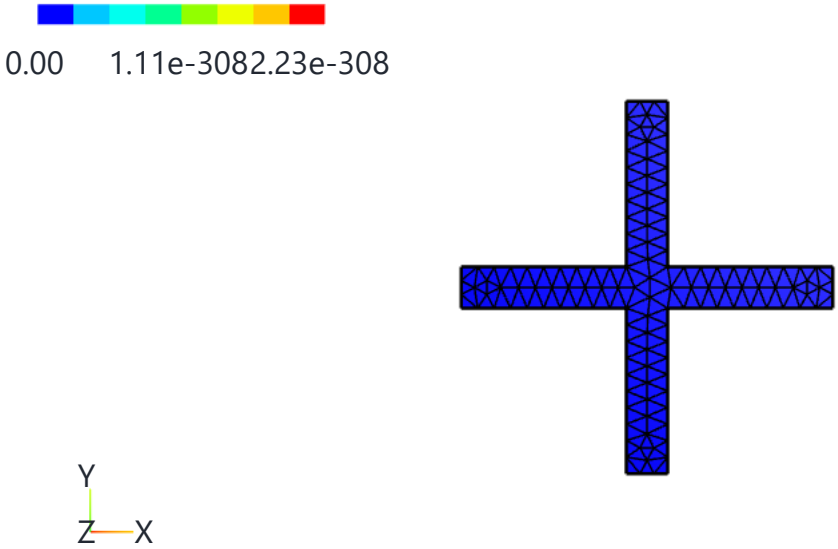
```
def KirchhoffLovePlateHHJ_Hyb(order=1):
    V = Discontinuous(HDivDiv(mesh, order=order - 1))
    Q = H1(mesh, order=order, dirichlet="clamped")
    H = NormalFacetFESpace(mesh, order=order - 1, dirichlet="clamped")
    X = V * Q * H
    (sigma, w, alpha), (tau, v, beta) = X.TnT()

    n = specialcf.normal(2)

    def tang(u):
        return u - (u * n) * n

    a = BilinearForm(X, symmetric=True, condense=True)
    a += (
        (
            InnerProduct(DMatInv(sigma, E, nu), tau)
            + div(sigma) * Grad(v)
            + div(tau) * Grad(w)
        )
```

```
            * dx
            - (sigma[n, :] * tang(Grad(v)) + tau[n, :] * tang(Grad(w)))
            * dx(element_boundary=True)
            + (sigma[n, n] * beta * n + tau[n, n] * alpha * n) * dx(element_boundary=Tr
        )
        a.Assemble()

        f = LinearForm(v * ds("force")).Assemble()

        gf_solution = GridFunction(X)

        f.vec.data += a.harmonic_extension_trans * f.vec
        gf_solution.vec.data += (
            a.mat.Inverse(X.FreeDofs(True), inverse="sparsecholesky") * f.vec
        )
        gf_solution.vec.data += a.harmonic_extension * gf_solution.vec
        gf_solution.vec.data += a.inner_solve * f.vec
        return gf_solution


    order = 1
    gf_solution = KirchhoffLovePlateHHJ_Hyb(order=order)
    gf_sigma, gf_w, _ = gf_solution.components

    Draw(gf_sigma, mesh, name="sigma")
    Draw(gf_w, mesh, name="displacement", deformation=True, euler_angles=[-60, 5, 30])
    Draw(gf_w.Operator("hesse"), mesh, name="hesse")
    print(
        "err w = ",
        sqrt(Integrate((gf_w - gf_w_ref) ** 2 + (Grad(gf_w) - Grad(gf_w_ref)) ** 2, mes
    )
```



NGSolve 6.2.2404-22-g3e237987d


-0.673
-0.22
0.237
Open Controls
Z
Y
X
NGSolve 6.2.2404-22-g3e237987d


0.00
1.11e-308
2.23e-308
Open Controls
Y
Z
X
NGSolve 6.2.2404-22-g3e237987d

```
err w =  0.020965099944146 26
```

8.3.3. Postprocessing property of HHJ method

We can use lowest order elements, i.e. linear polynomials for the vertical deflection w and piece-wise constants for the moment tensor σ. As $\sigma = \mathbb{D}\nabla^2 w$ one might try to compute a new quadratic $\tilde{w}$ as a postprocessing step element-wise with an improved order of convergence.

[Li. Recovery-based a posteriori error analysis for plate bending problems. *J Sci Comput.* 2021]

Correct displacement $w \in V_h^k$ with bubble functions of one degree higher $w_b \in V_h^{k+1}$ such that $\tilde{w} = w + w_b$

$$\tilde{w} = \arg \min_{\substack{v_h \in V_h^{k+1} \\ v_h|_{V_h^k} = w_h}} \|\mathbb{D}\nabla^2 v_h - \sigma\|_{L_2}^2.$$

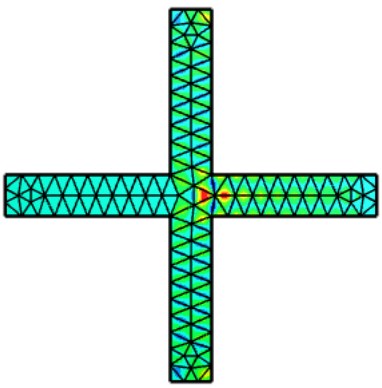
We need to solve a global problem. However, it was proved that the condition number of a Jacobi preconditioner stays bounded. Therefore, the costs are comparable to solve local problems.

```
from ngsolve.krylovspace import CGSolver

X = H1(mesh, order=order + 1, dirichlet="clamped")
w, dw = X.TnT()

freedofs = BitArray([False] * X.ndof)
for el in mesh.edges:
    freedofs[X.GetDofNrs(el)[order - 1]] = True
freedofs &= ~X.GetDofs(mesh.Boundaries("clamped"))


a = BilinearForm(
```

```
    InnerProduct(w.Operator("hesse"), dw.Operator("hesse")) * dx, symmetric=True
).Assemble()
f = LinearForm(
    InnerProduct(
        DMatInv(gf_sigma, E, nu) - gf_w.Operator("hesse"), dw.Operator("hesse")
    )
    * dx
).Assemble()

gf_wb = GridFunction(X)

preJpoint = a.mat.CreateSmoother(freedofs)
gf_wb.vec.data = CGSolver(a.mat, pre=preJpoint, printrates="", maxiter=100) * f.vec

Draw(gf_wb, mesh, "bubble_correction", order=3)
Draw(
    gf_w + gf_wb,
    mesh,
    "corrected_displacement",
    order=3,
    deformation=True,
    euler_angles=[-60, 5, 30],
)
print(
    "err w = ",
    sqrt(
        Integrate(
            (gf_w + gf_wb - gf_w_ref) ** 2
            + (Grad(gf_w) + Grad(gf_wb) - Grad(gf_w_ref)) ** 2,
            mesh,
        )
    ),
)
```

).000654 0.00027 0.00119

Open Controls



NGSolve 6.2.2404-22-g3e237987d

NGSolve 6.2.2404-22-g3e237987d

```
err w =  0.016180033423548238
```

Alternative post-processing: [Stenberg. Postprocessing schemes for some mixed finite elements. *ESAIM: M2AN 25* , 1 (1991), 151-167]

Solve the following problem element-wise

$$\widetilde{w} = \arg \min_{\substack{v_h \in P^{k+1} \\ v_h(V) = w_h(V)}} \|\mathbb{D}\nabla^2 v_h - \sigma\|_{L_2}^2$$

under the constraint that the vertex values are preserved. Therefore, a discontinuous Lagrange space can be used, where on each element the dofs corresponding to a vertex are

fixed.

- Advantage: Local problems
- Disadvantage: Corrected displacement can be discontinuous

Previous
8.2. Timoschenko and Euler-Bernoulli beam

Next
8.4. Reissner-Mindlin plate